Getting Started

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

CHAPTER 2
GETTING STARTED

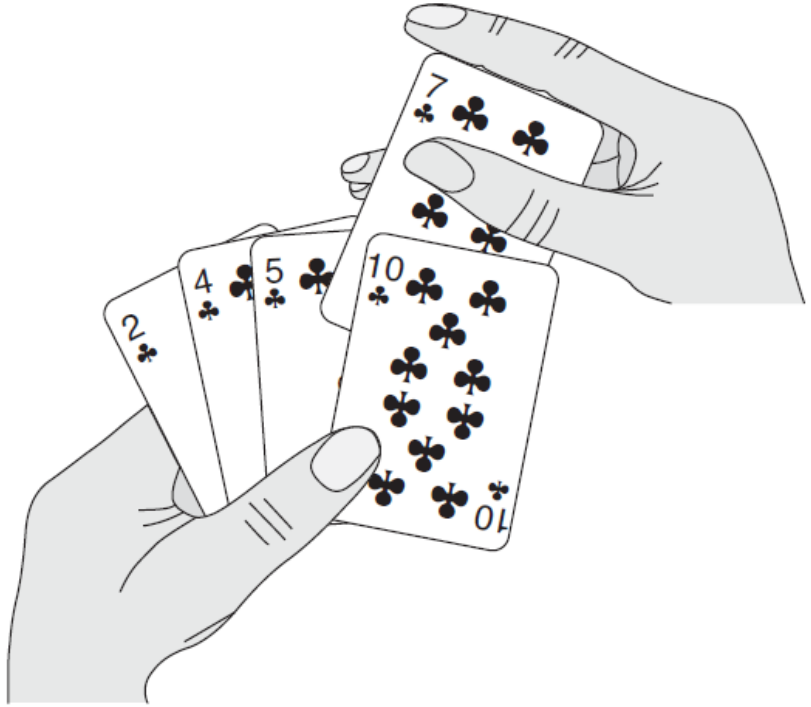
What is algorithm?

- ✓ Definition
  - A well-defined computational procedure that transforms
    - » input – some value, or set of values –
    - » into a desired output.
- ✓ Algorithm is a step-by-step description of a procedure which, if followed closely, produces a well-defined result.

Efficient in terms of execution time

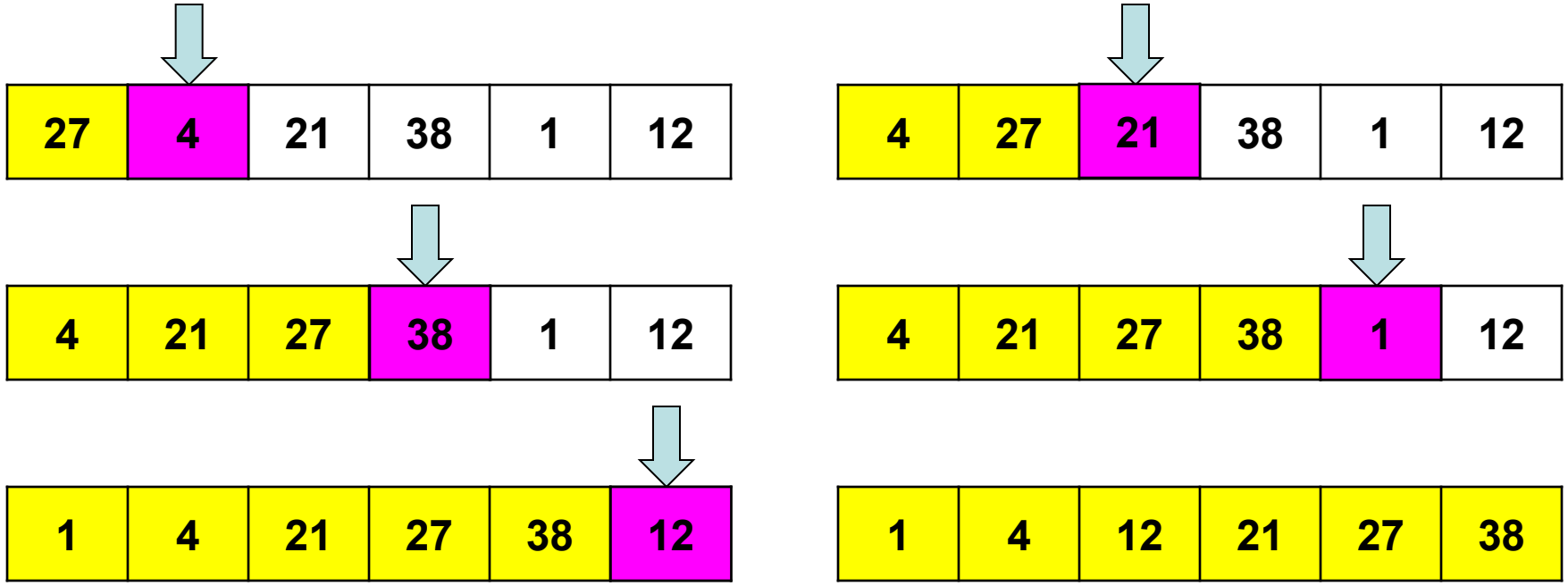
GETTING STARTED

- **We will begin with**
  - ✓ the example of '**`InsertionSort`**'
  - ✓ then take a look at the '**`MergeSort`**' briefly.



AN EXAMPLE : INSERTION SORT

Example)



An Example : Insertion Sort

1	2	3	4	5	6	
4	21	27	38	1	12	Key = 1, j = 5
4	21	27	38	38	12	i = 4
4	21	27	27	38	12	i = 3
4	21	21	27	38	12	i = 2
4	4	21	27	38	12	i = 1 → i = 0
1	4	21	27	38	12	

Insertion-Sort (*A*);

```
for j ← 2 to length(A)
    do  key ← A[j]
            ► Insert A[j] into the sorted sequence A[1..j-1].
            i ← j - 1;
            while i > 0 and A[i] > key
                do A[i+1] ← A[i];
                    i ← i - 1;
            A[i+1] ← key;
```

An Example : Insertion Sort

1	2	3	4	5	6	
4	21	27	38	1	12	Key = 1, j = 5
4	21	27	38	38	12	i = 4
4	21	27	27	38	12	i = 3
4	21	21	27	38	12	i = 2
4	4	21	27	38	12	i = 1 → i = 0
1	4	21	27	38	12	

Insertion-Sort (*A*);

for $j \leftarrow 2$ to *length(A)*
  do $key \leftarrow A[j]$
    ► Insert $A[j]$ into the sorted sequence $A[1..j-1]$.
    $i \leftarrow j - 1$;
    while $i > 0$ and $A[i] > key$
      do $A[i+1] \leftarrow A[i]$;
        $i \leftarrow i - 1$;
    $A[i+1] \leftarrow key$;

What are we going to learn?

- ✓ **Designing** the algorithm's approach
  - Design paradigms

- ✓ **Analyzing** the algorithm
  - Correctness
    - » An algorithm is said to be **correct** if, for every input instance, it halts with the correct output.
  - Efficiency
    - » Obtaining the time complexity of the algorithm

DESIGN PARADIGMS OF INSERTION SORT

- **Insertion-sort uses *incremental* approach**
  - ✓ having sorted the subarray $A[1 \ldots j-1]$,
    we insert the single element $A[j]$ into its proper place,
    yielding the sorted subarray $A[1 \ldots j]$

Correctness of Insertion Sort

- **Loop invariant**
  - ✓ At the start of each iteration of the for loop, the subarray $A[1 \ldots j-1]$ consists of the elements originally in $A[1 \ldots j-1]$ but in sorted order.
- **Initialization**
  - ✓ when $j = 2$, $A[1 \ldots j-1]$ consists of the single element $A[1]$.
  - ✓ Trivially sorted.
- **Maintenance**
  - ✓ Informally, the body of outer 'for' loop works by moving $A[j-1]$, $A[j-2]$, $A[j-3]$, and so on, by one position to the right until the proper position for $A[j]$ is found.
- **Termination**
  - ✓ The outer 'for' loop ends when $j = n+1$.
  - ✓ Thus, $A[1 \ldots n]$ consists of the elements originally in $A[1 \ldots n]$ but in sorted order.

This must have been covered by Discrete Mathematics!

```
Insertion-Sort (A);
  for j ← 2 to length(A)
    do key ← A[j]
       ► Insert A[j] into the sorted sequence A[1..j-1].
       i ← j - 1;
       while i > 0 and A[i] > key
          do A[i+1] ← A[i];
             i ← i - 1;
       A[i+1] ← key;
```

EFFICIENCY

- **Predicting the resources**
  - ✓ **Time, Space(memory)** that the algorithm requires **as a function of the input size n**
  - ✓ Space requirement
    - It was not a big deal but recently in the AI era it becomes a big deal.
  - ✓ Time requirement
    - in terms of the number of basic operations
  - ✓ The more **complex** an algorithm is, the more time and space it requires.

EFFICIENCY

- **Need to specify running time for a particular input size**
- **Input size depends on the problem**
  - ✓ Sorting n numbers
    - – number of items in the input
  - ✓ Multiplying two integers
    - – total number of bits needed to represent the input in ordinary binary notation.
  - ✓ Graph algorithms
    - – number of vertices and edges

- **Running time of an algorithm on a particular input**
  - ✓ It depends on the number of primitive operations or "steps" executed.
  - ✓ Each line(or operation/step) of pseudocode requires a constant amount of time.
  - ✓ Each line may take different amount of time.

Input size: 4-digit numbers

How many steps are required to find the right passcode?

TIME COMPLEXITY ANALYSIS

- **Worst-case: (usually)**
  - ✓ $T(n)$ = maximum time of algorithm on any input of size n.
- **Average-case: (sometimes)**
  - ✓ $T(n)$ = expected time of algorithm over all inputs of size n.
  - ✓ Need assumption of statistical distribution of inputs.
- **Best-case: (bogus)**
  - ✓ Cheat with a slow algorithm that works fast on *some* input.

TIME COMPLEXITY ANALYSIS

- **Usually, interested in the worst-case running time because**
  - ✓ It gives an upper bound
  - ✓ For some algorithms, the worst case occurs often.
  - ✓ Average case is often as bad as the worst case.

- **Average-case or expected running time**
  - ✓ Use **probabilistic analysis**
  - ✓ Need assumption about the distribution of the input.
  - ✓ **Randomized** algorithm : permute the input

```
Insertion-Sort (A);
    for j ← 2 to length(A)
        do  key ← A[j]
            ► Insert A[j] into the sorted sequence A[1..j-1].
            i ← j - 1;
            while i > 0 and A[i] > key
                do A[i+1] ← A[i];
                   i ← i - 1;
            A[i+1] ← key;
```

For j=2,…,n, let t_j be the number of times that the while loop is executed for that value j.

ANALYSIS OF INSERTION-SORT

- **Total time**
  $T(n) = ?$

- **What can $T(n)$ be?**
  - ✓ Best case - inner loop body never executed
    - $t_j = 1 \rightarrow T(n)$ is a linear function.
  - ✓ Worst case - inner loop body executed for all previous elements
    - $t_j = i \rightarrow T(n)$ is a quadratic function.
  - ✓ Average case
    - ???

4	21	27	38	1	12

4	21	27	38	1	12

Insertion-Sort (*A*);

```
for j ← 2 to length(A)
    do key ← A[j]
        ► Insert A[j] into the sorted sequence A[1..j-1].
        i ← j - 1;
        while i > 0 and A[i] > key
            do A[i+1] ← A[i];
                i ← i - 1;
        A[i+1] ← key;
```

For j=2,…,n, let t_j be the number of times that the while loop is executed for that value j.

MERGE SORT

```
// pseudocode
MergeSort(A, left, right) {
  if (left < right) {
    mid = floor((left + right) / 2);
    MergeSort(A, left, mid);
    MergeSort(A, mid+1, right);
    Merge(A, left, mid, right);
  }
}
```

Merge( )

takes two sorted subarrays of A and merges them into a single sorted subarray of A.

DESIGN PARADIGM

- ***Divide-and-conquer*** **approach**
  - ✓ A problem is divided into a number of sub-problems of smaller size to yield small results that can be combined to produce a solution to the original problem.
  - ✓ 3 steps
    - Divide
    - Conquer
    - Combine

MERGE SORT

- **Uses divide-and-conquer paradigm**
- **Divide**
  - ✓ divide the n-element sequence to be sorted into two subsequences of $n/2$ elements each.
- **Conquer**
  - ✓ sort the two subsequences recursively using merge sort.
- **Combine**
  - ✓ merge the two sorted subsequences to produce the sorted answer.

```
// pseudocode
MergeSort(A, left, right) {
  if (left < right) {
    mid = floor((left + right) / 2);
    MergeSort(A, left, mid);
    MergeSort(A, mid+1, right);
    Merge(A, left, mid, right);
  }
}
```

ANALYZING DIVIDE-AND-CONQUER ALGORITHMS

- Use a ***recurrence equation*** (or a ***recurrence***) to describe the running time of a divide-and-conquer algorithm.
- $T(n)$: running time on a problem of size n.
- If the problem size is small enough for some constant c ($n \leq c$), the straightforward solution takes constant time, $\Theta(1)$.
- a : number of subproblems
- n/b : input size of the subproblem
- $D(n)$: time to divide
- $C(n)$: time to combine

$\Theta(1)$ means it doesn't depend on n, i.e. the amount of time doesn't change even n increases or decreases.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

Analysis of merge sort

- **Divide**
  - ✓ Trivial
- **Conquer**
  - ✓ Recursively sort 2 subarrays.
- **Combine**
  - ✓ Linear-time merge.

$\Theta(n)$ means it linearly depend on n.

$$T(n) = 2T(n/2) + \Theta(n)$$

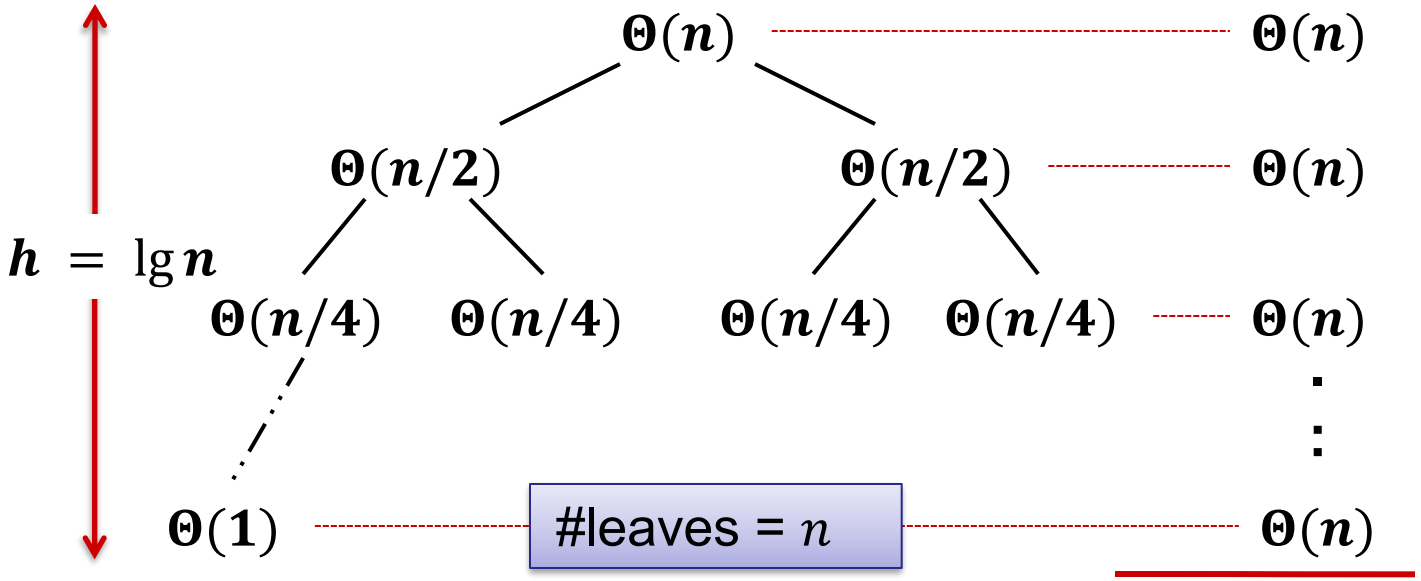
# of subproblems

subproblem size

Work dividing and combining

ANALYSIS OF MERGE SORT

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1; \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$



Total $= \Theta(n \lg n)$

No worries!
We will learn this again in the next class.

ANALYSIS OF MERGE SORT

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

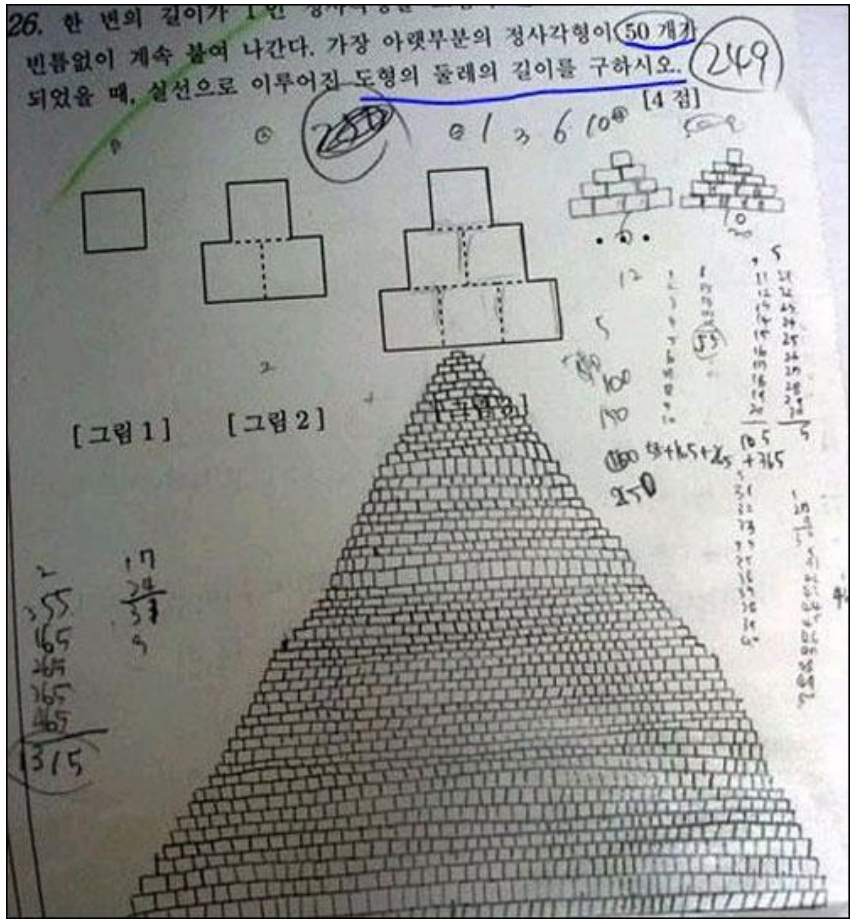
- By the master theorem (in Ch 4),
  we can show that $T(n) = \Theta(n \lg n)$.
- Compared to insertion sort ($\Theta(n^2)$ worst-case time),
  merge sort is faster.
- On small inputs, insertion sort may be faster.
  But, for large enough inputs, merge sort will always be faster, because its running time grows more slowly than insertion sort's.

KEYWORD

- **Designing Algorithm**
  - ✓ Design paradigms
- **Analyzing Algorithm**
  - ✓ Correctness : proof
  - ✓ Efficiency : time requirement
    - – Worst case
    - – Average case

OTHER DESIGN PARADIGMS

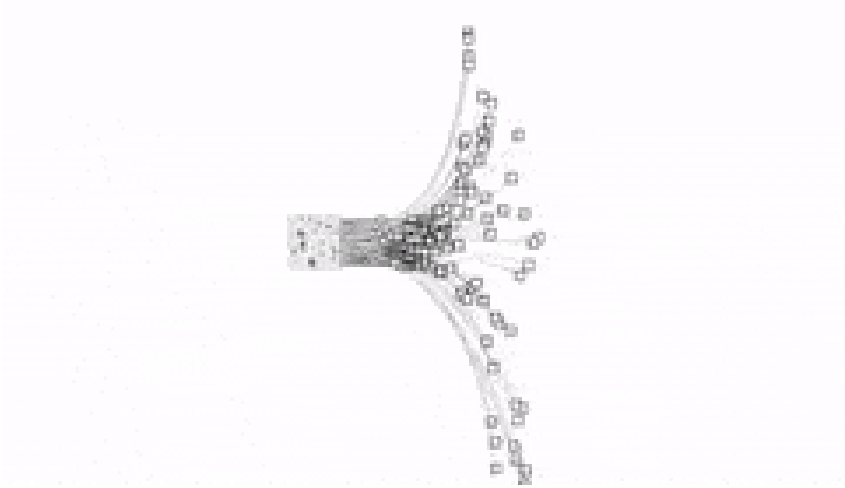
- Dynamic Programming
- Greedy
- Branch and Bound
- Brute force ?



OTHER DESIGN PARADIGMS

- Dynamic Programming
- Greedy
- Branch and Bound
- Brute force ?

$2.08 \times 10^{170} \gg 12 \times 10^{78}$



Thank you!